POSIX Storage

Access, through the portal, to data held on researcher-owned POSIX machines at INN and ImageTech. Identifiable data moves between those machines and is scrubbed there; only scrubbed data leaves. The portal is the interface: it decides who may do what and signs links. A small file service on each machine serves files to holders of those links. Where INN has administrator access, the operating system also enforces each link's grant; see Administrator access.

Requirements: POSIX storage (PX-1 to PX-15) and access control.

Status: built and tested locally on branch `posix/stack`; not deployed. See What isn't built yet.

Diagrams

Diagram	Shows	Checked against
Components	System overview; file service modules; backend POSIX modules	Python imports
Deployment	Processes, accounts, permissions on the machine	Dockerfile, entrypoint, the running container
Sequences	Download, resume, upload, delete, expiry	What each component logs, end to end
Temporary ID lifecycle	Free, Active, Quarantined, Retired	The helper's own records
Direct access alongside the portal	Who sets what on the files: administrators by hand, the portal per approval	Not drawn from code; each rule is tested (`tests_os/test_direct_access.py`)

Decisions

Date	Decision
2026-10-22	Portal-only access for now: browse, download, upload. Analysis through the portal is far off.
2026-10-22	One namespace per POSIX instance, used like a bucket name in paths, policies and the file tree (proposed; not yet confirmed).
2026-10-22	Links are reusable until they expire, so downloads can resume; expiry is checked when a download starts or resumes.
2026-10-23	With administrator access: temporary user IDs, and the web-facing service can't read the data, so the operating system enforces each grant.
2026-10-23	Downloads stream; nothing identifiable is pre-built into a stored copy.
2026-10-27	One temporary ID per approval, not per link (replaces one per link).
2026-10-28	Administrator access is used only where a requirement needs it (PX-8; proposed).
2026-10-29	Multi-file downloads are one ZIP streamed by the file service, never stored (PX-13, PX-14). Object storage keeps the bundler.

Administrator access

The aim (PX-8): the same design works on a machine where INN has no administrator access, and uses that access only where a requirement needs it. Each use below names the requirement or decision that would need it.

	Without administrator access	With it, added	Needed when
Who gets in through the portal	The portal checks access and signs a link per file; the file service checks each link.	Same.	Always; both ways.

	Without administrator access	With it, added	Needed when
What the file service can read	It runs as one ordinary account that can read the governed files, so a fault in it could reach any of them.	It can't open data files. A helper opens each requested file as the link's temporary ID, so the operating system enforces each grant.	No requirement yet. Proposed for identifiable data (PX-4); if kept, it becomes a requirement.
Who owns the governed files	The researchers, as today. They give the service's account read access (for example an ACL).	Still the researchers: administrators manage direct access (A1). The layout tool only closes the area to everyone else.	Always (PX-9).
Records of portal access on the machine	The file service's line per request (link ID, path, status); the person is in the portal's records.	The helper's line per operation, so the machine's own records trace each temporary ID to a person.	A2, if the machine's own records must show portal access (PX-10).
Records of direct work on the machine, such as scrubbing	Not possible.	The administrator sets up the machine's auditing.	A2, second and third answers (PX-10).
New files showing in the portal	A regular check of the machine.	Change reports as they happen (at least to set up).	A3, "within seconds" (PX-11).
Reaching the machine: port, firewall, TLS	The machine's administrators, either way.		Always.

Choosing per machine. One build runs either way, set by `GATEWAY_ACCESS`:

- `helper` (the default): the second column. The container starts the root helper and runs the web service as `gateway`, which refuses to start if it can read the data and never falls back.
- `account`: the first column. No root and no helper: the web service runs as the container's user, which must hold the data (it refuses to start if it can't read and write it), and the portal's links alone decide access. Run the container as that account (`user:` in compose).

Locally, `posix-gateway` runs the helper mode and `posix-gateway-account` (port 9102, namespace `inn-posix-account`) the account mode; the end-to-end tests cover both.

What enforces what

With administrator access (the tip of the branch).

The operating system enforces:

- Only the link's temporary ID can read the link's file: one ACL entry on that file. Folders are pass-through only (a shared group), never listable.
- The web service can't open any data file. It refuses to start if it can (it tries to read a canary file only the data owner can read), and answers 503 if the helper is unreachable. It never falls back to direct access.
- Uploads are written by the link's ID into a staging folder outside the data tree. Temporary IDs never get write access to data folders, where they could delete or replace other files.

The helper enforces, after checking the token itself:

- Final placement of uploads (moved into place through no-follow handles, given to the data owner with its folder's group and default entries, never open to everyone else).
- Deletes. POSIX can't grant "delete only this one file", so this can't be pushed down to the OS.
- Multi-file downloads: a file is opened for a bundle link only if it's on the list the link was signed for (the helper checks the list's SHA-256 against the link itself).

What that guarantees. The web service only ever receives descriptors for files a valid link names, opened by the helper as that link's ID. A compromised web service would still see the tokens and the bytes of files people actually request, but it couldn't reach anything nobody requested.

Also handled:

- Symlinks are never followed, at any step: paths are walked one component at a time with `O_NOFOLLOW`, and files are used through the handles that were checked. This closes the race where a folder is swapped for a symlink between a check and an open.
- Tokens are never logged. The file service writes its own access line (method, path, status, link ID) without the query string, and asks browsers not to pass links on (`Referrer-Policy: no-referrer`).
- Forged or altered tokens: RS256 only; unsigned tokens, HMAC with the public key, embedded keys, crafted key IDs and altered claims are all refused (tested).
- A file moved or replaced on the machine during a link's life can't reach a reused ID: grants record the file's device and inode, and an ID whose grant can't be revoked on the same file is retired.

Links and tokens

The portal backend signs every link (RS256; the key's PEM in `STORAGE_LINK_SIGNING_KEY`, or a file named by `STORAGE_LINK_SIGNING_KEY_FILE` for local stacks) and publishes the public key at `GET /api/storage-links/jwks.json`. A link is `{instance_url}/v1/files/{namespace}/{key}?token=…`. Claims: `iss` (`inn-portal`), `aud` (the namespace), `jti` (link ID), `iat`, `exp`, `op` (`read`, `write`, `stat`, `delete`), `path`, `sub` (the portal user the link was issued to), and optionally `grant` (the approval the link belongs to). The agreed contract, including URL encodings of awkward names, is `storage-gateway/contract/link-contract.json`; both the backend and the file service test against it.

Rotating the key. Each key's ID (`kid`) is its fingerprint. To rotate, move the old PEM to `STORAGE_LINK_SIGNING_KEY_PREVIOUS`, put the new one in `STORAGE_LINK_SIGNING_KEY` and restart the backend: new links are signed with the new key, and both public keys are published, so links issued before the rotation keep working until they expire. Once the longest-lived of them has expired (`BUNDLE_SOURCE_URL_EXPIRATION_SECONDS`, 2 hours by default, is the longest), remove the previous key.

Browser links use the query string; the backend's own calls (metadata, delete) send a short-lived token as `Authorization: Bearer`. Downloads support `Range` and `If-Range`; the validator changes when a file is replaced, rewritten or re-stamped (inode, change time, size, modification time).

Multi-file downloads

Several files download as one ZIP streamed by the file service, so nothing is built ahead or stored (PX-13, PX-14); object storage keeps the bundler. The portal checks access to every file, records the approved list (MongoDB `posix-bundles`) and returns one link, `{instance_url}/v1/bundles/{namespace}/{grant}`, whose token has `op: bundle`, the approval ID and `list`, the SHA-256 of the list as served. The file service fetches the list from `GET /api/storage-links/bundles/{grant}`, presenting that same link, and refuses a list whose digest doesn't match; with a helper, the helper checks it too. Entries are named `folder_name/namespace/key`, as the bundler names them. Entries are stored (not compressed) and ZIP64, so the length is exact and any byte range can be rebuilt: a broken download resumes with `Range` and `If-Range`, and the ETag changes if any file changes, so the download starts again. Checksums of files already sent are kept so a resume reads only what's left: in `GATEWAY_STATE_DIR` (a folder the file service can write to), where they survive a restart, or in memory if it's unset. The link lasts `BUNDLE_SOURCE_URL_EXPIRATION_SECONDS` (2 hours by default).

At most `GATEWAY_MAX_DOWNLOADS` (default 8) multi-file downloads stream at once per file service; another waits for a free slot, and after `GATEWAY_DOWNLOAD_WAIT_SECONDS` (default 5 minutes) is told to retry (503 with `Retry-After`). A slot is given back however a download ends. One download may list at most `GATEWAY_MAX_BUNDLE_FILES` (default 200,000) files; a longer list is refused (413).

Temporary IDs

Each approved request gets its own numeric user ID from a reserved range (locally 2,000,000 upward) the first time one of its links is used to read or upload. There is no account entry. Links of one request share an approval ID (`grant`), so a 10,000-file download uses one ID; a link without one is a request on its own. An approval belongs to the user of its first link (another user's link with the same approval ID is refused) and lasts until its latest link expires, so fresh links extend it; a link arriving after it expired starts a new ID. At expiry the sweeper removes the ID's ACL entries and staging files and quarantines the ID; it's reused only after the quarantine. States and transitions: the lifecycle diagram.

Audit

The helper writes one line per operation on the `storage-gateway.audit` logger, for example:

```
read link=28f87078… id=2000001 user=d30f195b-… path=inn-posix/study/scan.nii.gz grant=7c1e…
placed link=15c019e7… id=2000029 user=d30f195b-… path=inn-posix/e2e-2c702db6.bin
expired grant=7c1e… id=2000001 quarantined
```

So an operating-system audit record for a temporary ID traces to an approval, its links and a person. The path in these lines may itself be sensitive if file names identify participants (decisions document, Part D question 7).

Monitoring

`GET /v1/health` says whether the storage root is reachable. `GET /v1/metrics` serves request and error counters in the Prometheus text format: `posix_file_service_requests_total` by method, route and status code, and `posix_file_service_errors_total` (unhandled exceptions). Requests are counted by route, never by path or query, since a path names a file and the query carries the link's token. A request that fails while its body is streaming is counted with the status it started with. Neither endpoint needs a link; restrict them at the proxy if the counts shouldn't be public.

Working on the machine alongside the portal

Two ways reach the same files. People and their pipelines work on the machine with their own accounts, and administrators and project managers manage that access by hand over SSH (A1). The portal reaches files only through the helper, as a temporary ID per approval.

How the two meet on the files: Direct access alongside the portal.

	Set by administrators	Set by the portal
Owners and groups	Yes	Never changed (uploads get the folder's group)
ACL entries	Their own named entries	Only temporary-ID entries (user IDs from 2,000,000), added and removed per approval
Access for everyone else	None, kept that way by the layout tool	None
Folders	Whatever their groups need	Pass through only (`x` for the pass-through group), never list
New files and folders	Start closed to everyone else, from the folder's default entries	Uploads and folders they create take the folder's group and default entries

For administrators:

- Grant direct access with groups or ACL entries (`setfacl`); never open files to everyone else.
- Don't use the ACL mask to limit someone: each portal grant recomputes it. Put the limit in their own entry.
- Don't use user IDs in the temporary range, and never give the file service's account or the pass-through group read access.
- `python -m app.layout --root <area>` reports what differs from the above; `--apply` fixes it. It leaves owners, groups and administrators' entries alone, and reports symlinks, special files, multiply-linked files and leftover temporary entries for an administrator. Run it before first use, and any time to check.

Each of these is tested (`tests_os/test_direct_access.py`, `tests_os/test_layout.py`).

Running it locally

The main stack (`local/docker-compose.yaml`) must be up. Then, from the repository root:

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out backend/data/keys/storage-links.pem
docker compose -p ams-posix -f local/docker-compose.posix.yaml up -d --build
```

This adds the file service (`posix-gateway`, port 9100, reachable as `http://posix-gateway.localhost:9100` from both the host and containers), a backend built from this checkout (`ams-backend-posix`, port 8001) and Toxiproxy for fault tests. The second backend shares the main stack's Keycloak users and Postgres file tree, but keeps instances and policies in its own MongoDB database (`inn_mongo`; the main backend uses `ams_portal`), so POSIX test instances never land in the main stack's data. Data is on the `posix-data` Docker volume: Mac shared folders don't enforce Linux permissions or ACLs. Create a POSIX instance through the API on port 8001 (the UI on port 3000 talks to the main backend).

Tests

Suite	What	Where	Command (from `storage-gateway/`)
`tests/`	File service: examples, attacks, properties, stateful model, contract, fuzzing, diagrams vs imports, import rules	anywhere	`uv run --no-project --with-requirements requirements-test.txt pytest tests`
`tests_os/`	The OS boundary, helper protocol and fuzzing, layout tool, ID lifecycle diagram	Linux, root, ACL filesystem	`docker run --rm -v "$PWD/..:/repo" -w /repo/storage-gateway python:3.12-slim sh -c "pip install -r requirements-test.txt && pytest tests_os"`
`e2e/`	Through the portal: uploads, downloads, refusals, faults (Toxiproxy), sequence and deployment diagrams	local stack	`uv run --no-project --with pytest --with requests pytest e2e`
backend	Connector, signer, contract	backend image	`backend/tests/test_posix_agent.py`, `test_posix_contract.py`

Mutation testing (`mutmut`, configured in `storage-gateway/pyproject.toml`) checks that the tests notice small changes to the security-critical modules. The strategy, results and what's left are in the workstream's testing strategy.

How the diagrams are checked

Only Mermaid blocks whose first line is `%% check: <name>` are checked; others are explanation.

- **Module diagrams** (flowcharts): each node's label starts with the module's path in its component (`app/helper`, `core/storage_agents/posix_agent`). A plain solid arrow `A --> B` claims "A imports B". The test reads the imports from the source and fails if an arrow is missing or extra. Labelled or dotted arrows are runtime relationships and aren't import claims.
- **Deployment** (flowchart): labels state facts in a fixed form (`process uvicorn · uid 10001 · groups none`, `path /srv/posix/data · mode 0710 · owner 10002:10010`). They're compared with the Dockerfile and entrypoint, and with the running container.
- **Sequences**: a solid arrow `A->>B: METHOD /path → status` or `A->>Helper: op` is a claim. `{id}` and `{path}` stand for any ID or path; text in parentheses is explanation. The end-to-end test performs the operation and reads what the backend, file service and helper logged; since each logs a call when it finishes, the expected order is the diagram's completion order (a call made during another call finishes first). Dashed replies, async arrows and notes aren't checked.
- **Lifecycle** (state diagram): every `S1 --> S2` must be observed when the helper is driven through each case, and nothing else may occur.

When the code changes, a test fails until the diagram is updated, and the other way round.

Load

Measured 2026-10-31 with a helper (administrator access), in a Linux container on a 10-CPU laptop, using 1 KB files (`tests_os/test_scale.py`; the gate runs a 300-file version):

Multi-file download	Before batching	Now	Files per second now
10,000 files, one download	296 s	7.6 s	1,316
625 files each, 1 at once	19 s	0.4 s	1,416
625 files each, 4 at once	33 s	1.3 s	1,945
625 files each, 8 at once	59 s	3.4 s	1,457
625 files each, 16 at once	163 s	8.3 s	1,212

A multi-file download checks and opens its files 64 at a time (`OPEN_BATCH`): the helper grants each file and records the grants with one save, then starts one process as the temporary ID to open the batch, so the operating system still checks every file as that ID. Before, each file cost a process start (about 30 ms) and a rewrite of the helper's whole state, which grew with the square of the download's size. Files opened ahead of the stream are closed if it stops early; a file gone since it was listed fails when the stream reaches it. Throughput peaks around 4 downloads at once and holds past the spec's default cap of 8. Without a helper (`GATEWAY_ACCESS=account`) no process is started.

The measurement also found that the helper's listen backlog was Python's default of 5, so 8 downloads at once failed as unavailable; it's now the system's maximum.

Uploading a scanning session. A session is usually thousands of small DICOM files, uploaded from the technician's browser as a folder: one request plans the folder and signs a link per file, the browser sends 6 files at a time, and one request completes them (`e2e/test_upload_session.py`; the gate runs 200 files). 3,000 files of 512 KB (1.5 GB), with 5 ms of added latency each way:

Step	Time
Plan (one request)	12 s
Transfer (6 at a time)	96 s (16 MB/s)
Complete (one request)	10 s
Total	118 s, 39 ms per file

The cost is per file (each upload starts one process as the temporary ID to create its staging file), not per byte, and stays flat with size (36 ms per file for 200 files). Over fibre a session takes a few minutes, well within the links' 2 hours. The plan and complete steps are single requests that grow with the file count (about 4 ms per file each), so a folder of 15,000 or more files may meet a proxy's request timeout.

What isn't built yet

- For multi-file downloads: a link that outlives a long download (with a helper, each file is opened as the stream reaches it, so the link must still be valid then; how long links should last is undecided), read-ahead for many small files.
- Single-file downloads still get one approval per link; only multi-file downloads share one.
- Getting files already on the machine into the portal's file tree (file tracking, RDT's decision).
- Deployment to a real machine: hosting is RDT's decision, and it depends on the system administrators' answers (ID range, the helper's privileges, ACL support and limits, NFS settings, audit rules).